

The Cost of the Quantum Transition

ECE 4301 Final Project Report
Joshua Deckman, Jack Pearson

Introduction

Recent developments in quantum computing threaten the security of current, conventional encryption systems. Perhaps the area of cryptography that is in danger most is that of asymmetric encryption. According to [1], it is now theoretically possible for a quantum computer to break all “widely used public key systems,” which includes typical implementations of the Diffie-Hellman key exchange protocol. To solve this problem, a new type of asymmetric encryption system, called Kyber, has been developed that can, at least in principle, defend against attacks of quantum computers. However, for this increased security, Kyber must process more and send more data across a network in order to exchange symmetric keys. Thus, the goal of this project was to compare the performance of the Diffie-Hellman and Kyber systems across multiple networks to answer the question of whether the benefits of Kyber are worth their cost. This is accomplished in this project with a Raspberry Pi. Code was written for the Pi to simulate various network environments by regulating the Pi’s network stack. Through a number of simulations and tests, this project provides insight into the question at hand.

Implementation

Key Exchange Executable

A single binary executable was made to include functionality for both the client and the server in the public key exchange process. For both of these roles, the application, which was written in Rust, supports both the Diffie-Hellman and Kyber key exchange systems. Diffie-Hellman key exchange is supported in the project by the “x25519-dalek” Rust crate, while Kyber is supported by the “pqc_kyber” crate. When the executable is run as the server, it listens on a specified TCP port for incoming connections from a client. When run as the client, the application will initiate a key exchange with the server and measure the amount of time taken in the exchange. The client begins this process by sending a header packet of data to the server that specifies the exchange protocol to be used. The server then reads this packet to configure itself for the exchange. Once the key exchange process finishes, the client returns the exchange time to the console and exits. However, the server continues running, which allows a user to run another client with another key exchange protocol without having to restart the server.

In addition to the Kyber and Diffie-Hellman exchange protocols, a “buffer exchange” mode is also provided by the executable, which allows a user to have an arbitrary amount of data passed between the client and server in order to inspect the network channel over which an exchange takes place. The round-trip time for a packet of data is measured and returned by the client. This additional mode was used in the project to verify that custom network limitations that were imposed on the connection between client and server did in fact take effect.

Network Limiting

To simulate various network environments in which the Kyber and Diffie-Hellman protocols may be used, a bash script was written to regulate traffic through the network stack of the project’s Raspberry Pi. The script initiates a network namespace in which the client can execute.

The parameters that were used to regulate this network connection were the network bitrate and injected network latency. These parameters are specified at runtime. To simulate and acquire data for a wide range of networks, a test script was written to run multiple iterations of tests, sweeping across a range of values for these parameters.

Data Acquisition

The script to run test sweeps, which was written in Python, iteratively steps through a range of network rates and latencies. In addition to the bounds of iteration, a user can specify the step sizes between parameter values of successive iterations. On each iteration, the Python script executes two clients (one for Diffie-Hellman and one for Kyber) in the regulated network namespace and then parses their outputs for the key exchange times of the two protocols. The difference and ratio between these times are calculated, and the results stored. Once a test sweep completes, the results are written to a CSV file, which can be used for further data analysis.

Testing Procedure

For the project, testing was conducted in three phases. Initially, multiple buffer exchanges (using the buffer exchange mode of the executable) were run multiple times to (1) ensure that custom network limiting was properly imposed and (2) inspect overhead associated with the network. To do this, a buffer of 1,000 bytes was sent over a connection with a network rate set to 8 kilobits per second (kbps) and a latency set to 0 milliseconds. After this, 10,000 bytes were sent over an 80-kbps channel with no latency to inspect network overhead associated with the network rate. Finally, to observe the effect of adding latency to the connection, one byte was sent over a channel with 10 ms of delay (and a rate of 1 megabit, which hardly restricted the channel rate).

In order to obtain data that illustrates the effects of network regulation and delay most clearly, proper bounds on these test sweep parameters had to be found, which was the goal of the second phase of testing (which is herein referenced as “limit testing”). In this phase, multiple test sweeps were made, which elucidated bounds that were sufficient to clearly show the relationships between network latency, bitrate, and overall throughput.

The penultimate phase of testing included a test sweep between the bounds found in the second phase. Once the data from this sweep was acquired, it was then analyzed, as explained in the Results section of this report. Included in this analysis was the making of three-dimensional plots for visualizing the data. As might be expected, the three dimensions of these plots included network latency, network delay, and key exchange time. Through first these three phases of testing, results were acquired that are sufficient to illustrate the differences between the Kyber and Diffie-Hellman key exchanges over various network conditions.

A final evaluation was done running the simulation pointwise in conditions representative of common classes of networks: GSO satellite connections, IoT Zigbee connections, and best- and worst-case variants for internet connections.

Results

Buffer Exchange Results

For the first buffer exchange of 1,000 bytes and a network rate of 8 kbps, one might expect the 1,000 bytes (8,000 bits) to take exactly one second to reach the server and another second to return to the client. However, rather than taking 2,000 ms, the round-trip time was approximately

2960 ms, which indicated that there was a non-negligible amount of overhead associated with TCP communication. It turned out that this overhead decreased when a buffer of 10,000 bytes was sent over an 80-kbps channel. In this case, the round-trip time was about 2683 ms, indicating that overhead decreases as the packet size increases. Nonetheless, the overhead persisted.

When the effect of network latency was inspected with the 1-byte payload over the channel with a 10-ms delay, it was found that the total round-trip time was about 44 ms. It turns out that each normal data packet sent over TCP must be acknowledged by the recipient with an acknowledgement packet (ACK). Thus, instead of the expected 20 ms round-trip time for two packet communications, a time just above 40 ms was returned for the four messages that were passed (two data payloads and two ACKs). This is a significant contribution to the overhead associated with TCP.

Limit Testing Results

As limit testing was conducted, it became apparent that network channels with rates greater than 550 kbps had little effect on the throughput of the key exchange protocols. As discussed later, the key exchange time of each protocol, being inversely proportional to bitrate, approaches an asymptote. At 550 kbps, the key exchange time of each protocol was approximately equal to this asymptote, deviating little from this value for higher bitrates. Because of this, 550 kbps was selected as the upper limit for the main test sweep. During the sweep limit testing, it was also found that a lower bound of 50 kbps provided a sufficient range of data to capture the overall curve of the data. In addition to this, a step size of 50 kbps was selected for the sweep because it offered a suitable measurement resolution without requiring an excessive number of data points. Thus, the bitrate parameter values that were selected for the test sweep were 50 kbps for the lower bound on the channel bitrate, 550 kbps for the upper bound, and 50 kbps for the bitrate step between test iterations.

Concerning the network delay, it was discovered that, as is expected, the key exchange time is linearly proportional to the injected network delay. Consequently, the effect of injected channel delay was equally obvious for different test sweep bounds and step sizes of injected delay. Because of this, selecting the bounds and step size was somewhat arbitrary. The lower bound was chosen to be 0 ms so that the true throughput of each key exchange algorithm could be measured accurately for the case when the injected delay was 0 ms. Next, the upper bound was selected to be 40 ms because this amount of delay is noticeable, but not too significant to require a great deal of time for testing. Finally, a step size was selected so that the number of different latencies tested was equal to five, which would allow a sufficient number of test points for visualization, but not place an unnecessary load on the memory resources of the project's Pi. A step size that accomplishes this is 10 ms. Thus, the latency parameter values that were selected for the test sweep were 0 ms for the lower bound, 40 ms for the upper bound, and 10 ms for the step size.

Main Test Sweep Results

After the main test sweep was completed, the data was examined and plotted. These plots, and the raw data that was used to generate these plots, are given in the appendix. The three-dimensional graphs are plots of the exchange time of the respective protocol with respect to the latency and bitrate of the network. The raw data that was used to make these plots is given in Table 1 of the appendix. As shown in the figures, the exchange time of each system is inversely

proportional to the network rate, and linearly proportional with respect to injected latency (network delay). As the network rate increases, the exchange time decreases, approaching its asymptote (which occurs around 550 kbps). With a network delay of 0 ms, the value of this asymptote is approximately 30 ms for Kyber and 8 ms for Diffie-Hellman. Although the difference between these two values is only 22 ms, it is noticeable. In fact, Kyber takes about 3.7 times the amount of time taken by Diffie-Hellman.

One notable trend is shown in Figure 3, which gives the ratio of the exchange time of Kyber to that of Diffie-Hellman for no injected network latency. As shown, Kyber always takes about 3.8 times the amount of time that Diffie-Hellman takes, regardless of the network bitrate. However, even though this ratio almost stays the same, it does decrease to about 3.7 as the network bitrate increases to 550 kbps.

Pointwise Representative Test Results

We found that the post-quantum transition is most expensive in settings with low latency and also low bandwidth, such as Internet of Things applications, and therefore it is likely that the post-quantum transition will proceed most slowly in these domains. This is because Diffie-Hellman and Kyber have similar responses to network delay impairments, but Kyber is much more sensitive to low-bandwidth conditions because it has to transfer much more data.

Domain	Network Characteristics	Evaluation
GSO Satellite	10kbit—220mbit, 500ms delay	No difference at 8mbit (2s), small difference at 10kbit: ~2.4s vs ~3.5s. PQ transition is cheap
Zigbee (IoT)	1kbit—250kbit, 1ms delay	2kbit: 3.7s vs 9.5s; 20kbit: 242ms vs 823ms; 200kbit: 25ms vs 83 ms. PQ transition is expensive
Internet	512kbit—1gbit, 10-150ms delay	3G (512kbit, 150ms): 609ms vs 633 ms; Gigabit ethernet (1gbit, 10ms) 41 ms vs 43 ms. PQ transition is cheap

Conclusion

Clearly, the Kyber key exchange system is heavier than that of Diffie-Hellman. However, even though the performance of Kyber is worse than that of Diffie-Hellman, Kyber offers security against attacks from quantum computers. Diffie-Hellman does not. This begs the question, “Is it worth it?” Kyber does indeed take about 3.8 times the time as Diffie-Hellman, but with a network bitrate of at least 550 kbps, this delay is hardly noticeable. Consequently, the component of this study that most greatly affects the answer to whether Kyber is worth its cost is that of available network resources. If one has access to a connection that is faster than 550 kbps (which is not hard to come by), then, if Kyber can truly hinder quantum attacks, then it is likely worth spending the extra 22 ms to gain this security. Therefore, the conclusion to this study provides is the following: Although Kyber is heavier than Diffie-Hellman, this decrease in exchange speed is likely not noticeable for general applications. Kyber’s increase in security is well worth the cost in all but the most bandwidth-constrained, low-latency environments, such as very low-rate IoT links

References

- [1] R. Gonzalez. “Kyber - How does it work?” Approachable Cryptography.
<https://cryptopedia.dev/posts/kyber/> (accessed Dec. 8, 2025).

Appendix

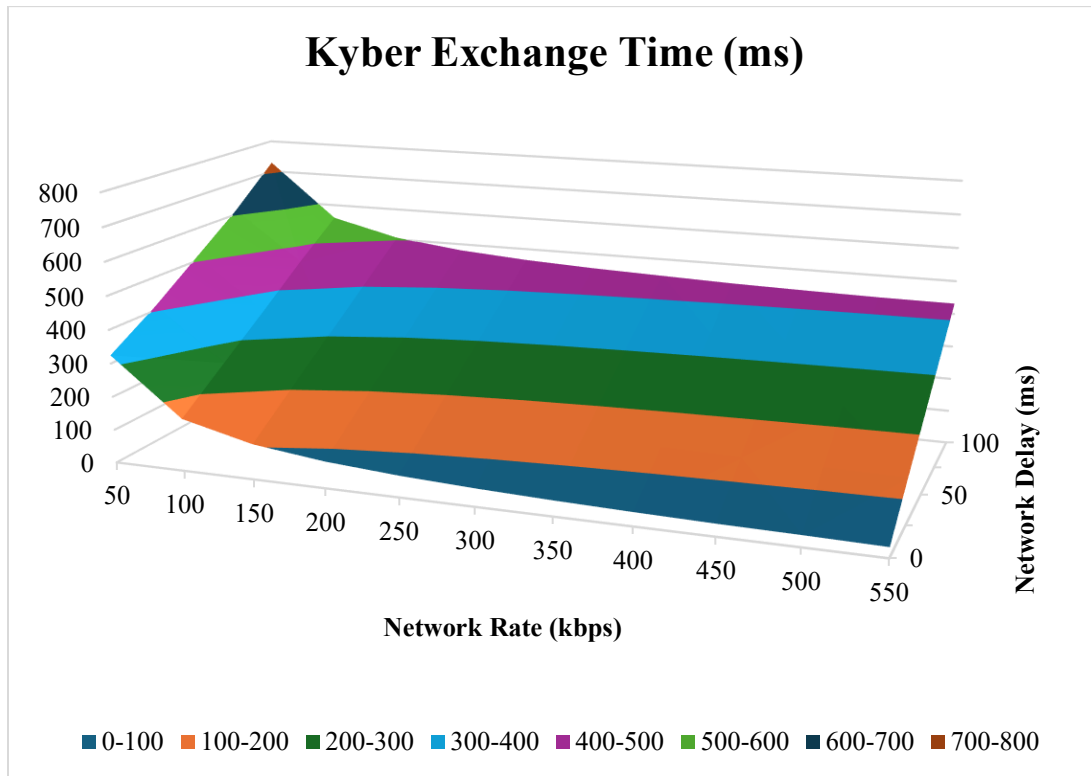


Figure 1: Key Exchange Times for the Kyber Protocol

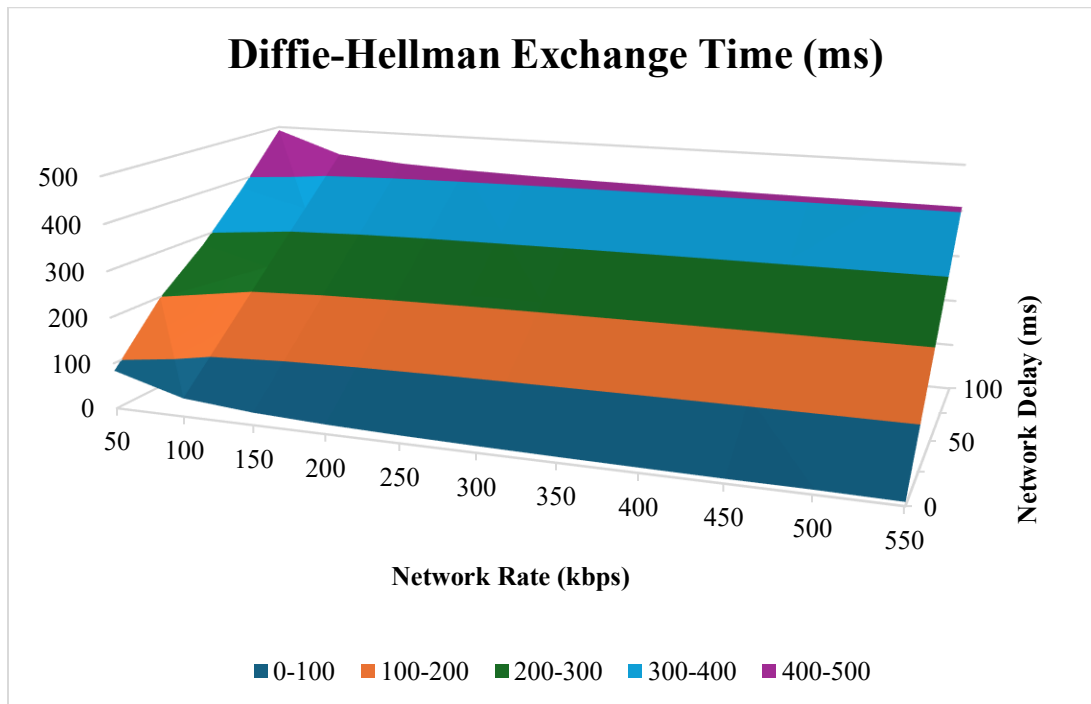


Figure 2: Key Exchange Times for the Diffie-Hellman Protocol

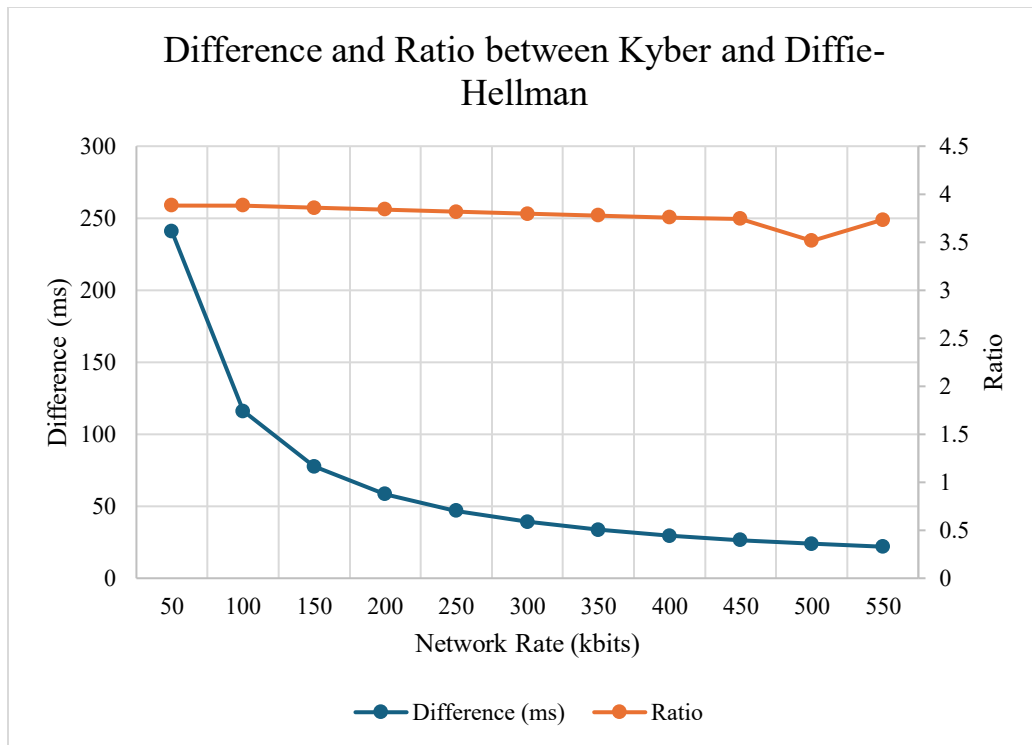


Figure 3: Difference and Ratio between Kyber and Diffie-Hellman Exchange Times

Table 1: Main Test Sweep Raw Data

Network Delay (ms)	Network Rate (kbits)	Kyber Exchange Time (ms)	Diffie-Hellman Exchange Time (ms)	Difference (ms)	Ratio
0	50	324.307	83.551	240.756	3.8815454
0	100	156.39	40.297	116.093	3.88093406
0	150	104.758	27.144	77.614	3.85934276
0	200	78.965	20.562	58.403	3.84033654
0	250	63.49	16.635	46.855	3.81665164
0	300	53.154	14	39.154	3.79671429
0	350	45.776	12.114	33.662	3.77876837
0	400	40.248	10.708	29.54	3.7586851
0	450	35.979	9.61	26.369	3.74391259
0	500	33.448	9.519	23.929	3.51381448
0	550	29.924	8.015	21.909	3.73349969
25	50	412.148	198.92	213.228	2.07192841
25	100	257.326	140.809	116.517	1.82748262
25	150	205.701	127.67	78.031	1.61119292
25	200	179.917	121.097	58.82	1.48572632
25	250	164.439	117.143	47.296	1.40374585
25	300	154.261	114.525	39.736	1.34696355
25	350	147.135	112.652	34.483	1.30610198
25	400	141.781	111.078	30.703	1.27640937
25	450	138.216	110.159	28.057	1.25469549
25	500	134.291	109.285	25.006	1.22881457
25	550	132.245	108.559	23.686	1.2181855
50	50	512.134	280.275	231.859	1.82725537
50	100	357.322	240.837	116.485	1.48366738
50	150	305.717	227.687	78.03	1.34270731
50	200	279.896	221.113	58.783	1.26585049
50	250	264.432	217.162	47.27	1.2176716
50	300	254.286	214.536	39.75	1.18528359
50	350	247.167	212.659	34.508	1.16226917
50	400	241.814	211.256	30.558	1.14464915
50	450	237.637	210.164	27.473	1.13072172
50	500	234.358	209.291	25.067	1.11977104
50	550	231.597	208.561	23.036	1.1104521
75	50	612.134	380.258	231.876	1.60978599
75	100	460.82	340.842	119.978	1.35200474
75	150	405.734	327.683	78.051	1.23819057
75	200	379.932	321.132	58.8	1.18310228
75	250	364.445	317.642	46.803	1.14734512
75	300	354.312	314.553	39.759	1.12639841
75	350	347.139	312.665	34.474	1.11025858
75	400	341.832	311.281	30.551	1.09814605
75	450	337.779	310.189	27.59	1.08894577
75	500	334.358	309.283	25.075	1.08107461
75	550	331.636	308.57	23.066	1.07475127
100	50	728.577	492.103	236.474	1.48053761
100	100	557.339	440.865	116.474	1.26419425
100	150	505.741	427.688	78.053	1.18249986
100	200	479.952	421.129	58.823	1.13967929
100	250	464.429	417.195	47.234	1.11321804
100	300	454.338	414.555	39.783	1.09596555
100	350	447.229	412.68	34.549	1.08371862
100	400	440.314	410.75	29.564	1.07197565
100	450	436.066	409.643	26.423	1.06450251
100	500	432.731	409.27	23.461	1.05732402
100	550	431.656	408.57	23.086	1.05650439

